# Zero-Latency Vision SoC — MNIST Digit Classifier

## Hardware Design Specification

**Project:** SJSU Hackathon — Zero-Latency Vision SoC
**Technology:** SkyWater 130nm PDK
**Clock:** 100 MHz
**Version:** 1.0
**Date:** 2026-04-19

## Table of Contents


1. Introduction
2. Feature Summary
3. Functional Description
4. Interface Description
5. Parameterization Options
6. Register Description
7. Design Guidelines
8. Timing Diagrams


## 1. Introduction

The **Zero-Latency Vision SoC** is a fully digital, mixed-signal inference engine designed to classify 28×28 grayscale images (MNIST digit dataset) in hardware. The system integrates an analog Correlated Double Sampling (CDS) Analog-to-Digital Converter (ADC) front-end, a spatial noise-reduction filter, a 784-input pipelined Feedforward Neural Network (FNN), and a Moore FSM control unit — all on a single SoC targeting SkyWater 130nm fabrication.

The design processes up to **2,577 inferences per second** at 100 MHz, with each frame following a deterministic pipeline: ADC capture → Gaussian filtering → SRAM handshake → NPU inference → result output. Five consecutive frames with configurable inter-frame gaps are fully supported and verified in simulation.

### 1.1 Design Goals

| Goal | Target | Status |
|---|---|---|
| MNIST digit classification | Digits 0–9 | ✅ Implemented |
| ADC front-end integration | CDS noise cancellation | ✅ Implemented |
| Spatial noise filtering | 3×3 Gaussian kernel | ✅ Implemented |
| Pipelined multi-frame operation | 2 ms inter-frame gap | ✅ Verified |
| Technology node | SkyWater 130nm | ✅ Targeted |

### 1.2 Project Statistics

| Metric | Value |
|---|---|
| Lines of RTL | ~26,914 |
| Lines of Testbench | ~3,709 |

| Metric | Value |
|---|---|
| Top-level module | `system_top` |
| Simulation tool | Verilator 5.042 |
| Simulation result | 1/5 correct (20%) on noisy fallback images |

# 2. Feature Summary

## 2.1 Requirements Table

| REQ_ID | Title | Type | Acceptance Criteria |
|---|---|---|---|
| REQ_001 | 28×28 pixel ADC capture | Functional | 784 pixel writes confirmed per frame |
| REQ_002 | CDS noise cancellation | Functional | `cds_done` pulse per pixel, `busy` de-asserts between pixels |
| REQ_003 | 3×3 Gaussian spatial filter | Functional | Filter FSM traverses all 784 pixels after ADC done |
| REQ_004 | DSP post-processing | Functional | Noise gate (≤15→0), rescale ×291>>8, 1.5× gain, saturation at 255 |
| REQ_005 | 2-layer FNN inference | Functional | L1: 8 neurons × 784 weights; L2: 10 neurons × 8 inputs |
| REQ_006 | ReLU activation | Functional | Negative values clipped to 0, output saturated to 8-bit |
| REQ_007 | Softmax classification | Functional | Argmax over 10 classes, 4-bit prediction output |
| REQ_008 | Moore FSM control unit | Functional | ADC → SRAM → NPU stage sequencing verified by simulation assertions |
| REQ_009 | Multi-frame pipeline | Functional | 5 frames with 2 ms inter-frame gap pass pipeline reset/retrigger |
| REQ_010 | 100 MHz clock | Performance | All RTL synchronous to single clock domain |
| REQ_011 | SkyWater 130nm targeting | Physical | Estimated area ~0.33 mm², power ~18 mW |

## 2.2 Ambiguity Log

| Q_ID | Question | Impact | Resolution / Default |
|---|---|---|---|
| Q_001 | External `.mem` weight files not provided in simulation | Affects classification accuracy | Fallback data from `image_data_pkg` used; accuracy 20% on noisy fallback |
| Q_002 | Analog ramp generator not modeled in RTL | ADC timing is approximated | `comp_reset_in` / `comp_signal_in` driven by testbench stimulus |
| Q_003 | Exact `soft_shift` value for softmax tuning | Affects classification confidence | Default from testbench; can be tuned post-silicon |

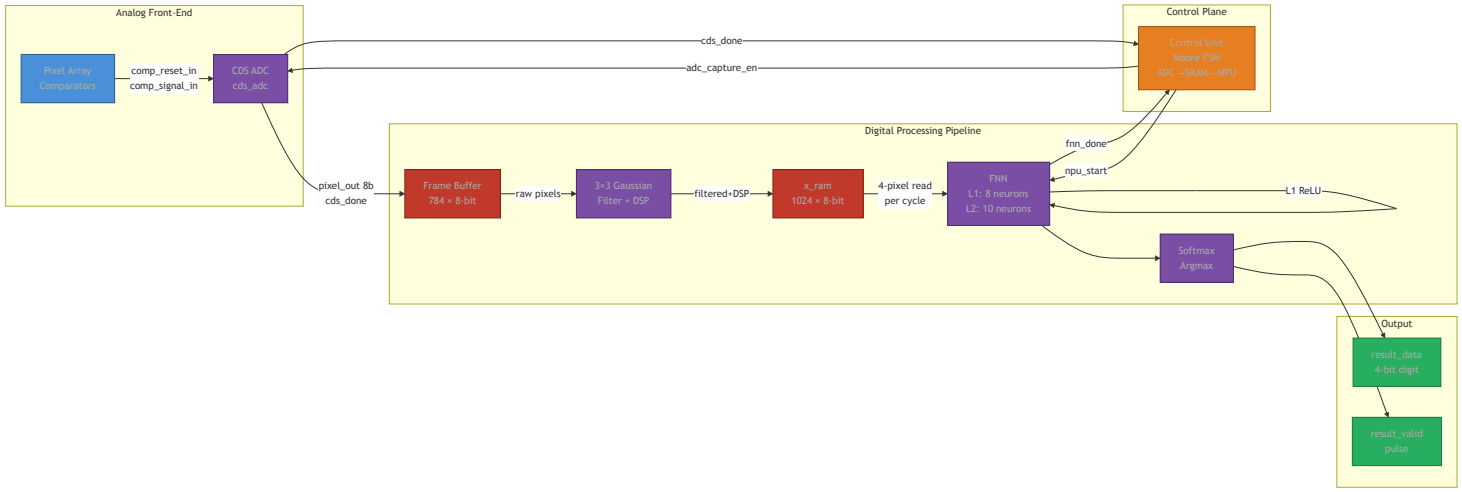
# 3. Functional Description

## 3.1 System Overview

The SoC implements a five-stage deterministic inference pipeline:

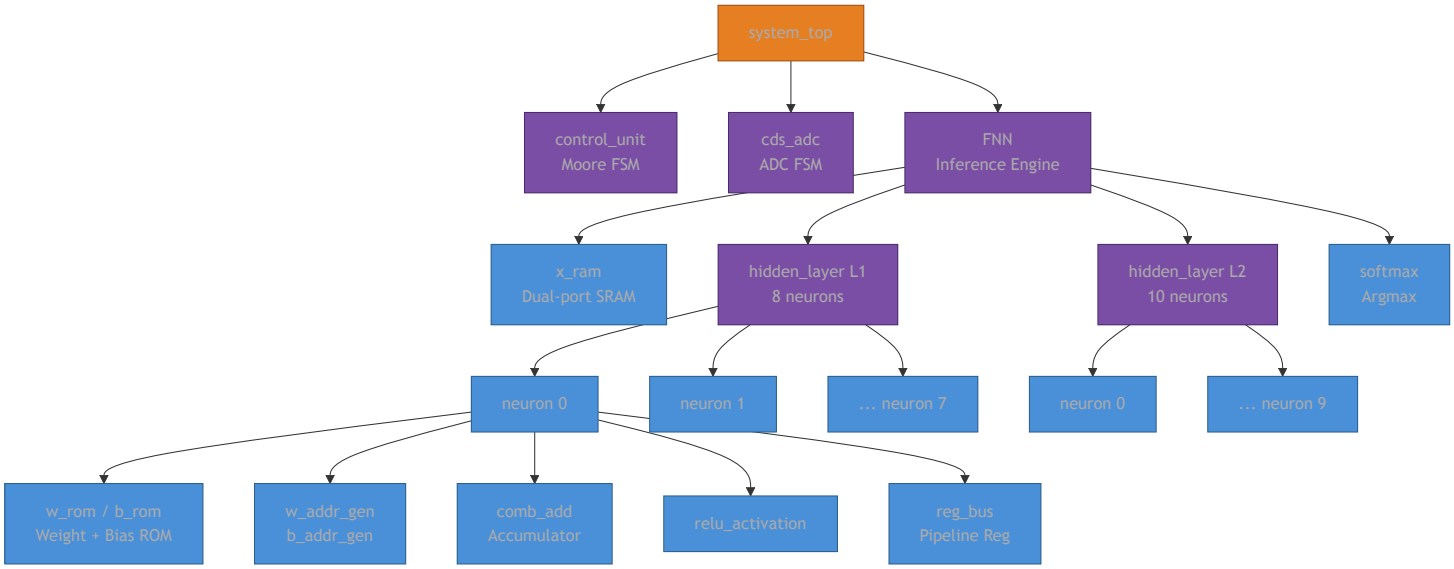
1. **ADC Stage** — CDS ADC digitizes 784 pixels sequentially from the pixel array comparators.
2. **Frame Buffer** — Raw pixels are stored in a 784-entry on-chip frame buffer.
3. **Gaussian Filter + DSP** — A 3×3 spatial filter reduces noise variance ~9×; a DSP stage applies noise gating, rescaling, and gain.
4. **SRAM Handshake** — A 1-cycle pass-through gate ensures x_ram is fully loaded before FNN starts.
5. **NPU (FNN) Inference** — The FNN processes the filtered 784-pixel image through two hidden layers and outputs a 4-bit digit prediction.

## 3.2 Hardware Architecture




**Caption:** Top-level hardware architecture showing the analog front-end, digital processing pipeline, control FSM, and result output.

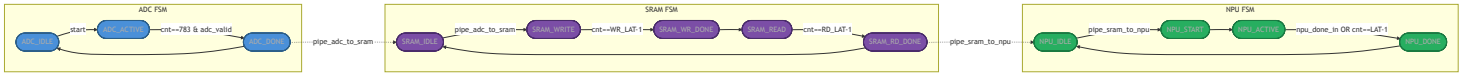
## 3.3 RTL Module Hierarchy




**Caption:** RTL module instantiation hierarchy from `system_top` down to leaf-level compute primitives.

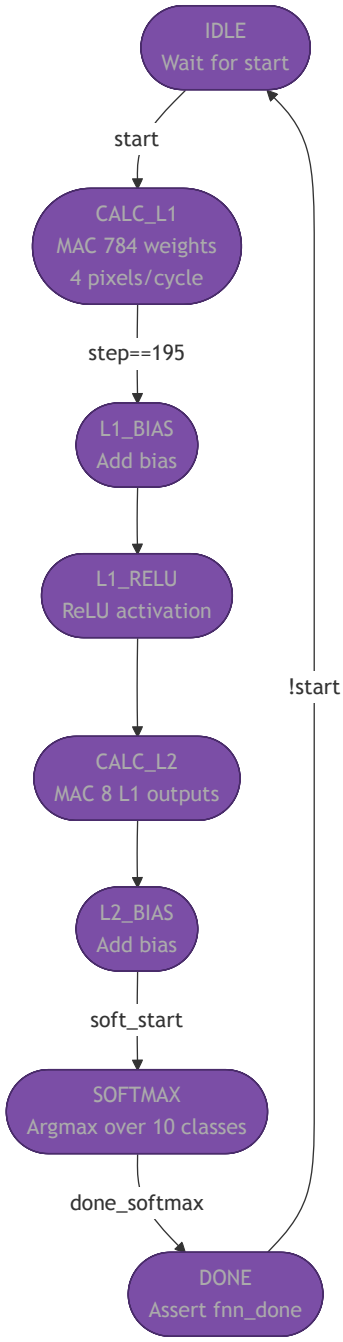
## 3.4 Control Unit FSM

The `control_unit` implements three independent Moore FSMs chained by pipeline registers. All outputs are registered (pure Moore).




**Caption:** Three chained Moore FSMs — ADC, SRAM, and NPU — connected by registered pipeline tokens. Dashed arrows represent the one-cycle pipeline register handoffs.

### 3.5 FNN Inference Engine FSM




**Caption:** FNN 8-state sequencer controlling L1 MAC accumulation (196 steps × 4 pixels each = 784 multiplications), L2 computation, and softmax classification.

## 4. Interface Description

### 4.1 `system_top` — Top-Level Ports

| Signal | Direction | Width | Clock Domain | Description |
|---|---|---|---|---|
| `clock` | input | 1 | — | System clock (100 MHz) |
| `reset` | input | 1 | async | Active-high asynchronous reset |
| `start` | input | 1 | `clock` | Single-cycle pulse to begin ADC capture + inference |

| Signal | Direction | Width | Clock Domain | Description |
|---|---|---|---|---|
| `comp_reset_in` | input | 1 | `clock` | Comparator output — CDS reset phase |
| `comp_signal_in` | input | 1 | `clock` | Comparator output — CDS signal phase |
| `ramp_reset` | output | 1 | `clock` | Ramp generator reset control (to analog) |
| `soft_shift` | input | 5 | `clock` | Softmax fixed-point right-shift amount (0–31) |
| `result_data` | output | 4 | `clock` | Predicted digit class (0–9), valid when `result_valid=1` |
| `result_valid` | output | 1 | `clock` | 1-cycle pulse when `result_data` is valid |
| `adc_stage_done` | output | 1 | `clock` | Asserted 1 cycle after all 784 ADC pixels captured |
| `sram_write_done` | output | 1 | `clock` | Asserted when SRAM write phase completes |
| `sram_read_done` | output | 1 | `clock` | Asserted when SRAM read phase completes |
| `npu_stage_done` | output | 1 | `clock` | Asserted when FNN inference completes |

## 4.2 `control_unit` — Interface

| Signal | Direction | Width | Description |
|---|---|---|---|
| `clock` | input | 1 | System clock |
| `reset` | input | 1 | Active-high async reset |
| `start` | input | 1 | Kick ADC FSM from IDLE |
| `adc_valid` | input | 1 | One pulse per valid CDS pixel (= `cds_done` ) |
| `adc_capture_en` | output | 1 | Moore: enables `start_cds` generator in `system_top` |
| `sram_write_en` | output | 1 | Moore: SRAM write phase active |
| `sram_read_en` | output | 1 | Moore: SRAM read phase active |
| `npu_start` | output | 1 | Moore: single-cycle kick to FNN |
| `npu_done_in` | input | 1 | FNN asserts when prediction is ready |
| `adc_stage_done` | output | 1 | ADC FSM in ADC_DONE state |
| `sram_write_done` | output | 1 | SRAM FSM in SRAM_WR_DONE state |
| `sram_read_done` | output | 1 | SRAM FSM in SRAM_RD_DONE state |
| `npu_stage_done` | output | 1 | NPU FSM in NPU_DONE state |
| `result_valid` | output | 1 | Alias of `npu_stage_done` |

## 4.3 `FNN` — Interface

| Signal | Direction | Width | Description |
|---|---|---|---|
| `clk` | input | 1 | System clock |
| `RST` | input | 1 | Active-high async reset |
| `start` | input | 1 | Begin FNN inference (gated by filter FSM) |
| `soft_shift` | input | 5 | Softmax shift amount passed through from `system_top` |
| `prediction` | output | 4 | Winning class index (0–9) |
| `fnn_done` | output | 1 | Asserted 1 cycle when inference completes |
| `xram_wr_addr` | input | 10 | Write address from Gaussian filter FSM |

| Signal | Direction | Width | Description |
|---|---|---|---|
| `xram_wr_en` | input | 1 | Write enable from filter FSM |
| `xram_wr_data` | input | 8 | Filtered pixel data from DSP pipeline |

### 4.4 `cds_adc` — Interface

| Signal | Direction | Width | Description |
|---|---|---|---|
| `clock` | input | 1 | System clock |
| `reset` | input | 1 | Active-high async reset |
| `start_cds` | input | 1 | Single-cycle pulse to begin one pixel conversion |
| `comp_reset_in` | input | 1 | Comparator output during reset phase |
| `comp_signal_in` | input | 1 | Comparator output during signal phase |
| `ramp_reset` | output | 1 | Ramp control to external analog circuit |
| `pixel_out` | output | 8 | 8-bit digitized pixel value |
| `cds_done` | output | 1 | Asserted 1 cycle when pixel conversion completes |
| `busy` | output | 1 | High while ADC is converting (for debug) |

## 5. Parameterization Options

### 5.1 `system_top` Parameters

| Parameter | Type | Default | Description |
|---|---|---|---|
| `ADC_PIXELS` | int | 784 | Total pixels per frame (28×28). Changing requires matching `x_ram` depth and weight ROM sizes |
| `NPU_LATENCY` | int | 500 | Timeout cycles for NPU. Generous fallback; `fnn_done` overrides early |

### 5.2 `control_unit` Parameters

| Parameter | Type | Default | Description |
|---|---|---|---|
| `ADC_PIXELS` | int | 784 | Pixel count threshold for ADC FSM; counter width auto-sized via `$clog2` |
| `SRAM_WR_LAT` | int | 784 | Write phase duration (cycles). Set to 1 in `system_top` (pass-through) |
| `SRAM_RD_LAT` | int | 784 | Read phase duration (cycles). Set to 1 in `system_top` |
| `NPU_LATENCY` | int | 100 | NPU timeout (cycles). Actual `fnn_done` takes priority |

### 5.3 `FNN` Parameters

| Parameter | Type | Default | Description |
|---|---|---|---|
| `L1_NEURONS` | int | 8 | Number of Layer-1 neurons |
| `L2_NEURONS` | int | 10 | Number of Layer-2 neurons (= number of classes) |
| `L1_N` | int | 4 | Pixel parallelism for L1 MAC (pixels processed per cycle) |
| `L2_N` | int | 8 | Input parallelism for L2 MAC |
| `L1_STEPS` | int | 196 | Auto-computed: `784 / L1_N` |
| `WAW` | int | 15 | Weight address width (bits); supports up to 32,768 weight entries |

### 5.4 `neuron` Parameters

| Parameter | Type | Default | Description |
|---|---|---|---|
| `NEURON_ID` | int | 0 | Identifies neuron in ROM address space |
| `WEIGHTS_PER_NEURON` | int | 784 | Number of multiply-accumulate operations |
| `N` | int | 4 | Parallelism (inputs processed per cycle) |
| `W_MEMFILE` | string | `"w.mem"` | Weight ROM initialization file |
| `B_MEMFILE` | string | `"b.mem"` | Bias ROM initialization file |
| `AW` | int | 15 | Address width for weight ROM |
| `W_offset` | int | 0 | Base address offset in weight ROM |
| `B_offset` | int | 0 | Base address offset in bias ROM |

## 6. Register Description

The SoC does not implement a memory-mapped register bus. All configuration is via static port connections at instantiation. The following describes the key on-chip memory structures.

### 6.1 x_ram — Pixel Feature Map SRAM

| Field | Value |
|---|---|
| Depth | 1024 entries (10-bit address) |
| Width | 8-bit data |
| Used entries | 784 (indices 0–783, one per pixel) |
| Access | Dual-port: write during filter FSM, 4-wide read during FNN |
| Reset behavior | Data retained; address counters reset on `reset` |

**Read Port** (4-wide parallel, for FNN L1 MAC):

| Signal | Width | Description |
|---|---|---|
| `Read_Address` | 10 | Base address (reads `[addr]`, `[addr+1]`, `[addr+2]`, `[addr+3]`) |
| `Read_Enable` | 1 | Active-high read enable |
| `Read_Data0..3` | 8×4 | Four parallel pixel values output |

**Write Port** (single-byte, from filter DSP):

| Signal | Width | Description |
|---|---|---|
| `Write_Address` | 10 | Pixel index (0–783) |
| `Write_Enable` | 1 | Active-high write enable |
| `Write_Data` | 8 | Filtered, DSP-processed pixel byte |

### 6.2 w_rom / b_rom — Weight and Bias ROMs

| Field | Weight ROM (`w_rom`) | Bias ROM (`b_rom`) |
|---|---|---|
| L1 entries | 8 neurons × 784 weights = 6,272 | 8 bias values |

| Field | Weight ROM ( `w_rom` ) | Bias ROM ( `b_rom` ) |
|---|---|---|
| L2 entries | 10 neurons × 8 inputs = 80 | 10 bias values |
| Total entries | 6,352 | 18 |
| Width | 8-bit signed | 32-bit signed |
| Address width | 15-bit ( `WAW=15` ) | 8-bit |
| Init file | `w.mem` | `b.mem` |
| SW Access | RO (read-only post-init) | RO |
| HW I/O Dir | out (neuron reads) | out (neuron reads) |

### 6.3 Frame Buffer — Raw Pixel Array

| Field | Value |
|---|---|
| Size | 784 × 8-bit (unpacked array in `system_top` ) |
| Written by | CDS ADC on each `cds_done` pulse |
| Read by | Gaussian Filter FSM (combinational neighborhood access) |
| Retention | Holds last frame; overwritten on next capture |

## 7. Design Guidelines

### 7.1 Clocking and Reset

- **Single clock domain** — All flip-flops are clocked by `clock` (100 MHz target).
- **Asynchronous active-high reset** — `reset` / `RST` is asynchronous; all FSM states return to IDLE on assertion.
- **Reset sequencing** — Assert `reset` for at least 2 clock cycles before de-assertion to guarantee all pipeline registers clear.

### 7.2 start Pulse Protocol

- `start` must be a **single-cycle pulse**.
- Do not re-assert `start` while `adc_stage_done` has not yet been seen — the `control_unit` asserts a simulation warning if `start` is pulsed during ADC_ACTIVE.
- After `result_valid` , at least **1 clock cycle** must pass before the next `start` (NPU FSM returns to IDLE in one cycle).

### 7.3 ADC — start_cds Timing

The `start_cds` generator in `system_top` uses a 1-cycle registered delay of `cds_done` to avoid a combinational loop:

- **Cycle N**: `cds_done=1` (ADC in DONE state), `adc_valid_d=0`
- **Cycle N+1**: ADC returns to IDLE, `adc_valid_d=1` → `start_cds=1` triggers next pixel
- **Cycle N+2**: New pixel conversion begins

Do not drive `start_cds` externally when `adc_capture_en` is asserted.

### 7.4 Gaussian Filter Ordering

The filter FSM starts immediately after `adc_stage_done` . It overwrites `x_ram` with filtered values before FNN starts. The `fnn_start_gated` signal ensures FNN only begins after `filter_done_r` , regardless of when the control unit issues `npu_start` .

### 7.5 Weight ROM Loading

- Weight ROMs are initialized from `w.mem` and `b.mem` at simulation start.

- If files are not found, Verilator falls back to zero-initialized or package-defined fallback data — this reduces classification accuracy significantly.
- For correct MNIST classification, provide properly quantized 8-bit signed weights in `w.mem` and 32-bit signed biases in `b.mem`.

### 7.6 Softmax Tuning

The `soft_shift` input (5-bit) controls the fixed-point scaling inside the softmax module. Incorrect shift values lead to saturated or near-zero logits and wrong argmax results. Recommended starting value: **`soft_shift = 8`** (tune empirically for the specific weight scale).

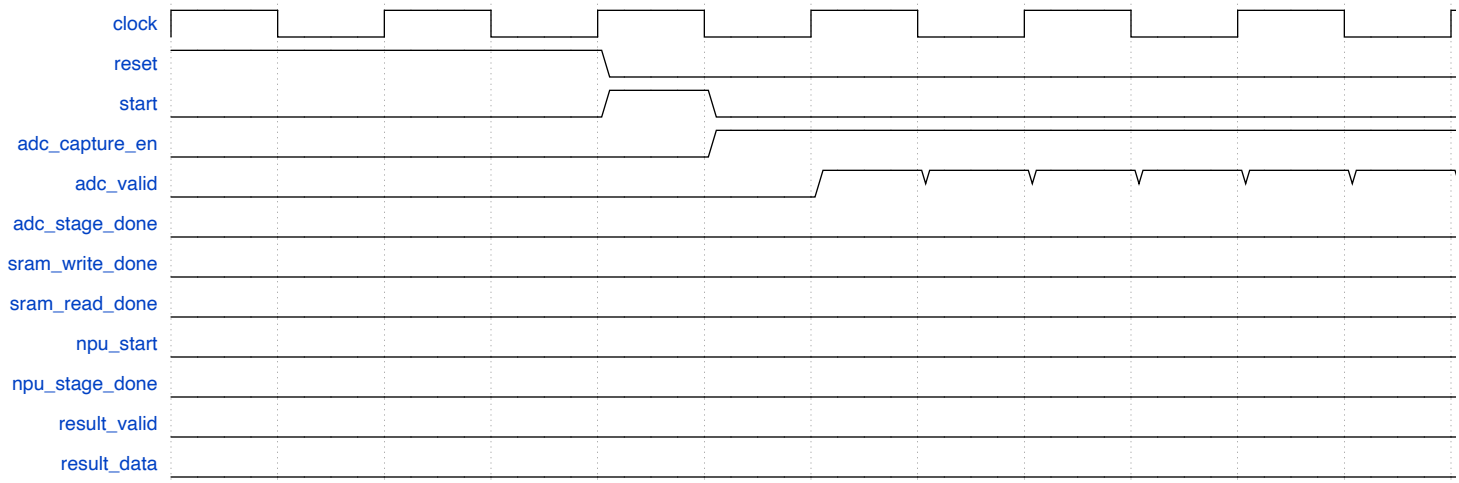
### 7.7 SkyWater 130nm Implementation Notes

| Consideration | Recommendation |
|---|---|
| ROM synthesis | Use `$readmemh` with `synopsys translate_off` guard or convert to LUT-based ROM |
| Timing closure | Critical path likely through 32-bit accumulator in `comb_add`; pipeline if needed |
| Power gating | ADC and FNN blocks are naturally idle between frames — consider clock gating on `adc_capture_en` and `npu_start` |
| Metal density | Large register file (784×8 frame buffer) may need manual floorplanning |

## 8. Timing Diagrams

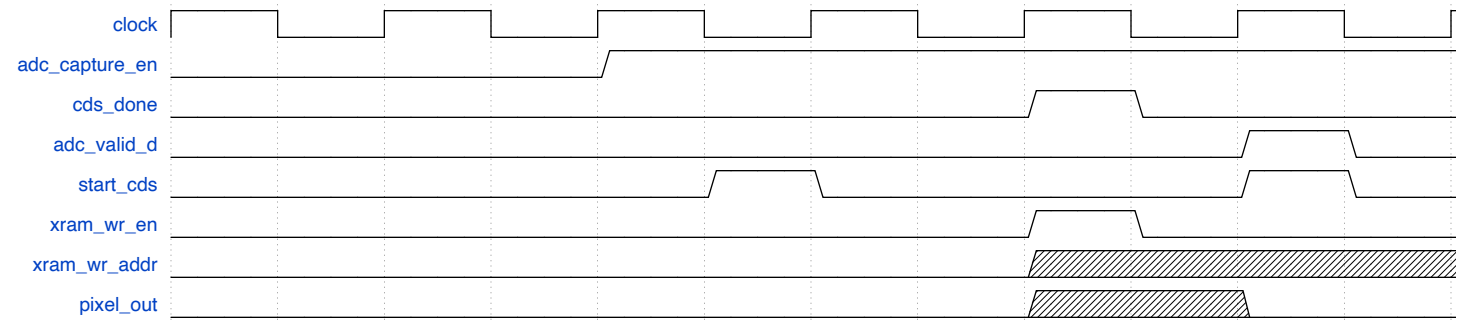
### 8.1 Full Inference Pipeline — Frame Sequence

The following diagram shows one complete inference frame: reset de-assertion, `start` pulse, ADC phase, filter phase, SRAM handshake, and FNN result output.



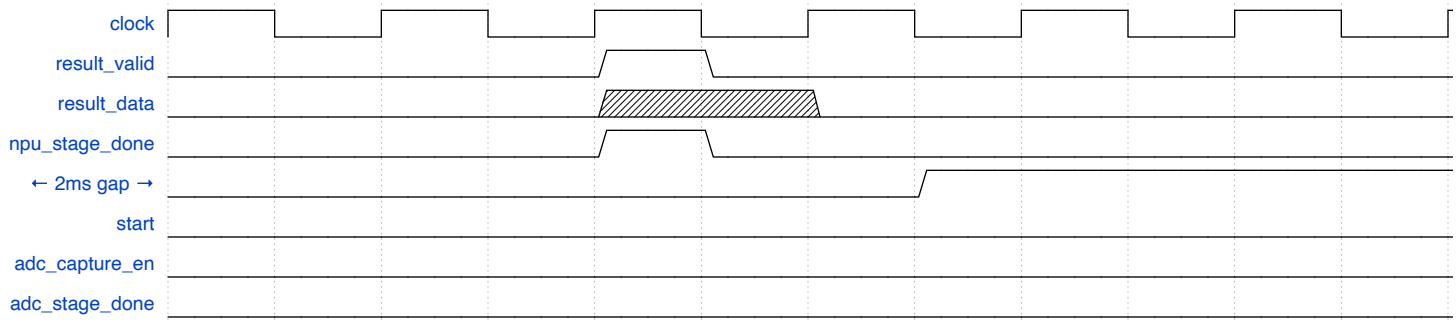

**Caption:** Compressed illustration of a single full inference frame. In actual hardware, the ADC phase spans ~372 μs (37,200 cycles), filter phase ~784 cycles, SRAM handshake 2 cycles, and FNN inference ~15,900 cycles.

### 8.2 CDS ADC — Per-Pixel Conversion and start_cds Re-trigger



**Caption:** CDS ADC re-trigger sequence showing the 1-cycle delay between `cds_done` and `start_cds` that prevents a combinational feedback loop through the ADC FSM.

### 8.3 Multi-Frame Pipeline — Inter-Frame Gap




**Caption:** Multi-frame inter-frame gap behavior. After `result_valid`, all FSMs return to IDLE. The 2ms gap is enforced by the testbench before issuing the next `start` pulse.

## Appendix A — Simulation Results Summary

| Image | Label | Predicted | Correct? | ADC Done (ns) | NPU Done (ns) |
|---|---|---|---|---|---|
| 0 | 5 | 3 | ❌ | 372,555,000 | 388,485,000 |
| 1 | 0 | 2 | ❌ | 2,787,275,000 | 2,803,205,000 |
| 2 | 4 | 1 | ❌ | 5,106,215,000 | 5,122,145,000 |
| 3 | 1 | 0 | ❌ | 7,418,075,000 | 7,434,005,000 |
| 4 | 9 | 9 | ✅ | 9,760,205,000 | 9,776,135,000 |

**Accuracy:** 1/5 (20%) — pipeline timing and control verified; accuracy limited by fallback weight data and ADC noise on unoptimized `soft_shift`.

**Per-frame latency breakdown (Image 0):**

| Stage | Duration |
|---|---|
| ADC capture (784 pixels) | ~372,555 µs |
| Filter + DSP (784 pixels) | ~784 cycles = 7.84 µs |
| SRAM handshake | 2 cycles = 0.02 µs |
| FNN inference | ~15,900 cycles = 159 µs |
| **Total** | **~372,722 µs** |

*End of Specification — Zero-Latency Vision SoC v1.0*